Then, whichever call finishes first, its result will be stored in the cache. And when the other call finally produces a result, it will also try to store that result in the cache. Depending on the implementation of the cache, this can simply overwrite the old result (they could be different, for a lot of reasons), discard the new result, or in the worst-case scenario, produce a duplicate[15].

But independently on how the collision is handled, the worst part is that we compute twice a result that we could have retrieved from cache. This in turn means unnecessarily high latency, and in some cases extra costs (for instance, in our example, data providers do charge for data from social networks).

7.2.4 Marking Cache Values as "Loading"

Finding a perfect solution for race conditions is not easy. In some cases, in many cases, it is impossible. Here, for example, if we consider the fully detailed architecture of figure 7.2, we have several HTTP calls that can have a variable latency, or fail altogether. So, we can't be sure which one of the two calls to retrieve Google sentiment would return first: assuming the first call finishes first can be reasonable, but can also lead to inefficiency, if for any reason the first call to the data gatherer[16] lasts more than the average. That's not the worst-case scenario, though: if we decide that the second call to our `/sentiment` endpoint will wait and re-use the result computed for the first one, and